RAJARSHI SHAHU COLLEGE OF ENGINEERING
Tathawade, Pune – 411033
Department of Computer Engineering

MINI PROJECT REPORT

MOVIE / OTT RECOMMENDATION AND TREND ANALYSIS

An Integrated Analytical System Using Fact Constellation Warehousing, Apriori Association Rules, Multi-Modal Classification, Regularized Regression, and ARIMA Time Series

Subject:	Data Mining and Analytics
Subject Code:	CS3203T-A
Academic Year:	2026–2027
Semester:	1

Submitted by:	**Vinit Shinde**
Roll No:	37
PRN:	RBT24CS178
Branch:	B.Tech – Computer Engineering
Faculty Guide:	Prof. Jit Shendekar

Autonomous Institute Affiliated to Savitribai Phule Pune University • Academic Year 2026–27

CONTENTS

Sec	Title	Sec	Title
1	Abstract	8	Methodology and Algorithm
2	Introduction	9	System Architecture
3	Problem Statement	10	Software and Tools
4	Aim and Objectives	11	Experimental Output and Evaluation
5	Scope of the Project	12	Limitations and Future Scope
6	Syllabus Mapping	13	Conclusion
7	Dataset Description	14	References

1. ABSTRACT

With the rapid expansion of digital streaming media platforms, entertainment distributors face fundamental challenges in catalog discovery, customer churn reduction, and acquisition planning. This mini-project presents an end-to-end analytical data mining architecture that unifies theatrical viewer preferences from the **MovieLens 100k** dataset (100,836 ratings across 9,742 movies) with commercial video-on-demand distribution patterns from the **Netflix Titles** catalog (8,807 streaming records). The system executes an automated, idempotent ETL pipeline loading raw records into a conformed **Fact Constellation Data Warehouse** in SQLite across 5,340 conformed dates with complete relational foreign key integrity.

Upon this multidimensional warehouse, five complementary data mining modules are executed: (1) Apriori market-basket mining discovering 935 high-lift co-viewing association rules (Lift up to 6.15x); (2) Multinomial Naive Bayes natural language processing classifying plot summaries into genres with 50.49% accuracy; (3) Balanced Random Forest predicting movie rating tiers with 63.01% accuracy without data leakage; (4) Regularized Lasso regression performing sparse feature selection (Test RMSE: 0.4683); and (5) Box-Jenkins ARIMA(3,1,3) time series modeling achieving a 22.47% forecast improvement over naive persistence. Finally, a weighted hybrid recommender engine evaluated offline via Leave-One-Out Hit-Rate@10 (0.0467) and served through an interactive Flask web application provides practical, explainable deployment.

2. INTRODUCTION

Modern media distribution relies extensively on data mining algorithms to extract actionable intelligence from vast, heterogeneous consumer interaction logs. Online streaming services generate continuous transaction feeds, descriptive metadata, and temporal catalog releases. Examining these multidimensional datasets manually is infeasible due to extreme sparsity and combinatorial complexity. Data mining bridges this analytical gap by uncovering latent associations between titles, classifying unstructured narrative synopses, projecting temporal catalog growth, and personalizing viewing queues.

By integrating traditional market-basket analysis (Apriori) with statistical time series forecasting (ARIMA), text mining (TF-IDF vectorization), and supervised ensemble learning (Random Forest), content providers can reliably predict audience reception, optimize procurement budgets, and deliver transparent, explainable recommendations to subscribers without black-box opacity.

3. PROBLEM STATEMENT

To ingest, cleanse, and structure heterogeneous movie rating logs and streaming catalog metadata into an analytical Fact Constellation warehouse, and to apply association rule mining, text classification, predictive regression, and ARIMA temporal modeling so that co-viewing affinities, rating tiers, and platform catalog trends can be reliably discovered, evaluated, and served through an explainable recommendation interface.

4. AIM AND OBJECTIVES

Aim: To design, develop, and empirically evaluate an integrated Movie and OTT Recommendation and Trend Analysis system using multidimensional warehousing and core data mining algorithms.

- Construct an idempotent ETL pipeline to assemble a SQLite Fact Constellation schema with conformed date dimensions and 0 orphan keys.
- Formulate user viewing records into transactional baskets and apply Apriori to mine frequent itemsets and co-viewing rules.
- Develop a text mining pipeline using NLTK Porter Stemming, TF-IDF vectorization, and Multinomial Naive Bayes to classify plot synopses.
- Engineer leak-free metadata features and train Decision Tree and Random Forest classifiers to predict movie rating categories.
- Train and benchmark Linear, Ridge, and Lasso regression models to estimate continuous movie ratings and analyze active L1 feature weights.
- Conduct ADF stationarity tests and fit ARIMA models on monthly additions to forecast catalog expansion with 95% confidence intervals.
- Synthesize Content-based, Item-Item Collaborative Filtering, and Apriori rules into an explainable Hybrid Recommender (evaluated via Hit-Rate@10).
- Deploy an interactive Flask web application and Chart.js analytical dashboard for real-time inference.

5. SCOPE OF THE PROJECT

The project encompasses end-to-end data processing, from raw CSV cleansing to analytical warehousing, model training, offline mathematical evaluation, and interactive dashboard deployment. All reported metrics reflect empirical test evaluations on fixed splits without data leakage. The project operates on benchmark historical catalogs (MovieLens 100k and Netflix Titles); it does not establish causal human behavior nor perform automated live video transcoding.

6. SYLLABUS MAPPING

Syllabus Unit / Topic	Implementation in this Project	Outcome
Unit 1: Warehousing & Preprocessing	Fact Constellation schema in SQLite (warehouse.db), conformed dim_date, OLAP queries.	**CO1**
Unit 2: Transformation & Reduction	Regex release year extraction, genre one-hot encoding, log transform, Porter Stemming, TF-IDF.	**CO1, CO2**
Unit 3: Regression Analysis	OLS, Ridge (L2 shrinkage), Lasso (L1 feature selection), Decision Tree Regressor for continuous ratings.	**CO3, CO4**
Unit 4: Association Rules & Classification	Apriori algorithm (support/confidence/lift), Multinomial Naive Bayes, Decision Trees, Random Forest.	**CO2, CO4**
Unit 5: Advanced Mining & Recommenders	ADF stationarity differencing (d=1), ARIMA(3,1,3) forecasting, Content-based TF-IDF, Item-CF, Hit-Rate@10.	**CO5**

7. DATASET DESCRIPTION

Dataset File	Records / Shape	Raw Attributes	Transformation & Cleansing
ratings.csv	100,836 rows (610 users)	userId, movieId, rating, timestamp	Converted epoch to UTC ISO date; mapped rating tiers: Low (<3), Med (3-3.9), High (>=4). 0 orphan keys.
movies.csv	9,742 movies	movieId, title, genres	Extracted 4-digit year via regex; split pipe genres into conformed dim_genre; cleaned titles.
tags.csv & links.csv	3,683 tags / 9,742 links	userId, movieId, tag, imdbId	Cleaned text, lowercased, merged for content-based TF-IDF document profiles.
netflix_titles.csv	8,807 titles	show_id, type, director, cast, country, date_added, duration, description	Stripped whitespace, parsed date_added into conformed dates, parsed duration, split multi-genres into bridge table.

8. METHODOLOGY AND ALGORITHM

Step 1 – Ingestion & Discovery: Robust automated path discovery scanning nested directories, reading raw CSVs into DataFrames.

Step 2 – Fact Constellation ETL: Automated idempotent relational pipeline loading 11 normalized tables into SQLite with primary/foreign keys.

Step 3 – Basket Assembly & Apriori Mining: Formulated 599 users × 300 movies binary co-viewing matrix. Extracted frequent itemsets and association rules using mlxtend.

Step 4 – Text Mining & Naive Bayes: Plot synopsis cleaning, Porter stemming, TF-IDF vectorization (20,000 max features), and MultinomialNB tuning.

Step 5 – Rating Tier Classification: Engineered 22 leak-free features (19 genre flags, release year, log ratings, tag counts) and trained Random Forest.

Step 6 – Regularized Regression: Evaluated Linear, Ridge, and Lasso regression models with cross-validation to analyze L1 coefficient sparsity.

Step 7 – Time Series Differencing & ARIMA: Stationarity verification via ADF unit-root test (d=1), grid search over (p,1,q), and 12-month holdout evaluation.

Step 8 – Hybrid Recommender & Serving: Linear ensemble of Content-based, Item-CF, and Apriori rules served through a responsive Flask web app.

Key Measures and Mathematical Formulations

Measure / Algorithm	Mathematical Formula / Objective Function	Interpretation in Project Context
Support	$\text{support}(A \rightarrow B) = P(A \cup B) = \text{Count}(A \cup B) / N$	Fraction of user viewing baskets containing both titles.
Confidence	$\text{confidence}(A \rightarrow B) = P(A \cup B) / P(A)$	Conditional probability that a user likes title B given they liked A.
Lift	$\text{lift}(A \rightarrow B) = \text{confidence}(A \rightarrow B) / \text{support}(B)$	Affinity strength: Lift > 1 indicates positive association above independence.
Naive Bayes	$P(c \mid d) \propto P(c) \prod P(w_i \mid c)$	Posterior genre probability given word tokens in plot synopsis.
Lasso (L1)	$\min (1/2n)\|\|y - X\beta\|\|^2 + \lambda\Sigma\|\beta_j\|$	L1-penalized regression driving uninformative genre weights to exactly 0.
ARIMA(p,d,q)	$\phi(B)(1-B)^d y_t = \theta(B)\varepsilon_t$	Autoregressive integrated moving average for monthly OTT additions.
Hybrid Score	$S_h = 0.40\ S_{content} + 0.30\ S_{cf} + 0.30\ S_{rule}$	Dynamic linear combination with weight normalization for missing signals.

9. SYSTEM ARCHITECTURE

Pipeline Stage	Component & Implementation Mechanism	Generated Artifact
1. Warehouse ETL	SQLite Fact Constellation schema with 11 relational tables & 11 indexes	warehouse.db
2. Association Engine	mlxtend Apriori frequent itemsets on 599x300 co-viewing matrix	models/rules.joblib
3. Classification	MultinomialNB (genre text) & Balanced Random Forest (rating tier)	models/*_classifier.joblib
4. Trend & Regression	LassoCV rating predictor & statsmodels ARIMA(3,1,3) monthly additions	models/arima_models.joblib
5. Serving Engine	Hybrid Recommender + Flask REST API + Jinja2 + Chart.js dashboard	app.py / web interface

10. SOFTWARE AND TOOLS

Core Language: Python 3.12 • **Data Handling:** Pandas 2.3.3, NumPy 2.5.2 • **Database:** SQLite 3 (ACID-compliant Fact Constellation) • **Machine Learning:** Scikit-Learn 1.6.1 • **Pattern Mining:** mlxtend 0.25.0 • **Time Series:** statsmodels 0.15.0 • **NLP:** NLTK 3.10.4 • **Web & UI:** Flask 3.1.3, Jinja2, Chart.js • **Testing:** Pytest 9.1.1 (19/19 passing), Playwright 1.64.0.

11. EXPERIMENTAL OUTPUT AND EVALUATION

All metrics reported below were produced by the automated training pipeline (python train_all.py) on reproducible holdout splits:

Module	Algorithm	Primary Metric	Secondary Metric	Optimal Parameter
Genre Text Mining	Multinomial Naive Bayes	**Accuracy: 50.49%**	Macro F1: 0.4757	$\alpha = 0.1$
Genre Baseline	Logistic Regression	Accuracy: 51.54%	Macro F1: 0.5215	C = 1.0
Rating Tier Class	Decision Tree	Accuracy: 50.68%	Macro F1: 0.4766	max_depth = 10
Rating Tier Class	**Random Forest**	**Accuracy: 63.01%**	Macro F1: 0.5576	n_est=200, depth=10
Rating Regression	**Lasso (L1)**	**Test RMSE: 0.4683**	Test R^2: 0.3105	$\alpha = 0.00041$
Rating Regression	Ridge (L2)	Test RMSE: 0.4684	Test R^2: 0.3103	$\alpha = 10.0$
Time Series ARIMA	ARIMA(3,1,3) on Netflix	**Test RMSE: 41.42**	Naive RMSE: 53.42	**+22.47% vs. Naive**

Recommendation Method	Hit-Rate @ 10	Precision @ 10	Recall @ 10	Top Discovered Apriori Rule
Content-Based (TF-IDF)	0.0067	0.0007	0.0067	**Kill Bill: Vol. 1 → Kill Bill: Vol. 2**
Apriori Association Rules	0.0267	0.0027	0.0267	Support: 10.85% • Confidence: 73.86%
Item-Item Collaborative	**0.0467**	0.0047	0.0467	**Lift: 6.15x (Strongest Rule)**
Hybrid Ensemble Engine	**0.0467**	**0.0047**	**0.0467**	The Godfather & Goodfellas → Part II (Lift: 4.43x)

12. LIMITATIONS AND FUTURE SCOPE

Limitations: (1) The MovieLens 100k catalog spans 1996–2018 theatrical releases whereas Netflix additions span 2008–2021, reflecting divergent distribution channels. (2) Netflix synopses are concise (~25 words), constraining lexical separation for subtle genres. (3) Natural user rating skew required class-balanced loss weighting during classification.

Future Scope: (1) Transition from cosine similarity to Matrix Factorization (SVD) and Neural Collaborative Filtering (NCF). (2) Ingest streaming clickstream telemetry (completion rate, pause points) via Kafka. (3) Containerized Kubernetes microservices deployment with continuous retraining.

13. CONCLUSION

This mini-project successfully engineered an integrated Data Mining and Analytics architecture fulfilling all course outcomes (CO1–CO5). An automated ETL pipeline structured 100,836 ratings and 8,807 streaming titles into an analytical SQLite Fact Constellation warehouse. Association mining mined 935 high-lift co-viewing rules, text mining classified plot synopses, Random Forest predicted rating tiers with 63.01% accuracy, Lasso regression isolated critical genre features, and ARIMA(3,1,3) forecasted monthly platform additions with a 22.47% error reduction over baseline persistence. The hybrid recommender and responsive Flask dashboard demonstrate how multidimensional warehousing and core data mining algorithms unite into a practical production system.

14. REFERENCES

1. Agrawal, R., Imielinski, T., & Swami, A. (1993). *Mining Association Rules between Sets of Items in Large Databases*. ACM SIGMOD.
2. Box, G. E., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control* (5th ed.). Wiley.
3. Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.
4. Harper, F. M., & Konstan, J. A. (2015). *The MovieLens Datasets: History and Context*. ACM TiiS, 5(4), 1–19.
5. Tibshirani, R. (1996). *Regression Shrinkage and Selection via the Lasso*. J. R. Statist. Soc. B, 58(1), 267–288.
6. Savitribai Phule Pune University • CS3203T-A Data Mining and Analytics Course Curriculum.

Submission Verification: Submitted by Vinit Shinde (Roll No: 37, PRN: RBT24CS178). All metrics verified via train_all.py test runs.